TALLER PRÁCTICO

Premier League Data API

Guía de Uso | Machine Learning I (ML1-2026I)

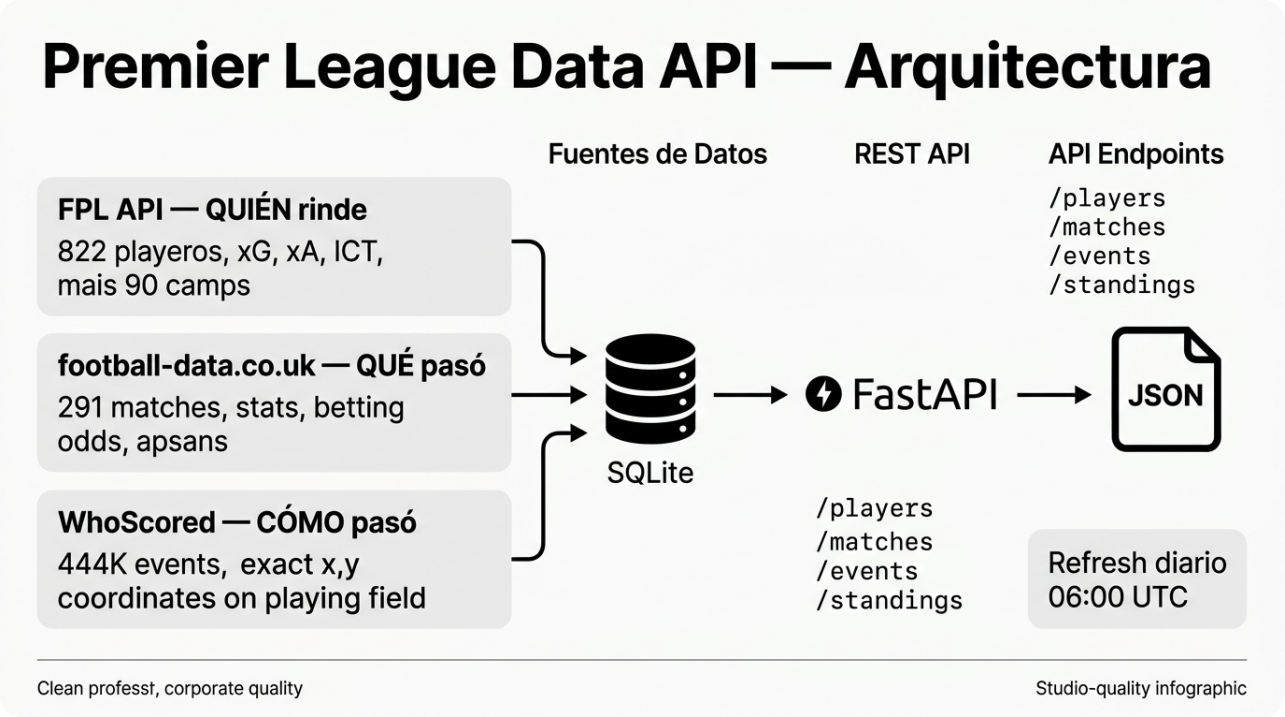
Julián Zuluaga — Docente

15 de marzo de 2026

Guía de Uso: Premier League Data API

Introducción

La **Premier League Data API** es un servicio REST que consolida tres fuentes de datos de la Premier League 2025-26 en endpoints JSON listos para consumir desde Python. Diseñada para el curso Machine Learning I, esta API elimina la necesidad de scraping, parsing de CSVs o manejo de autenticación — los estudiantes simplemente llaman `requests.get()` y obtienen datos estructurados.



Arquitectura: tres fuentes de datos unificadas en una API REST con actualización diaria

Acceso a la API

Recurso	URL
Base URL	`https://premier.72-60-245-2.sslip.io`
Swagger UI	`https://premier.72-60-245-2.sslip.io/docs`
Documentación LLM	`https://premier.72-60-245-2.sslip.io/llms.txt`
OpenAPI Spec	`https://premier.72-60-245-2.sslip.io/openapi.json`

No requiere autenticación. Todos los endpoints son públicos y de solo lectura.

Fuentes de Datos

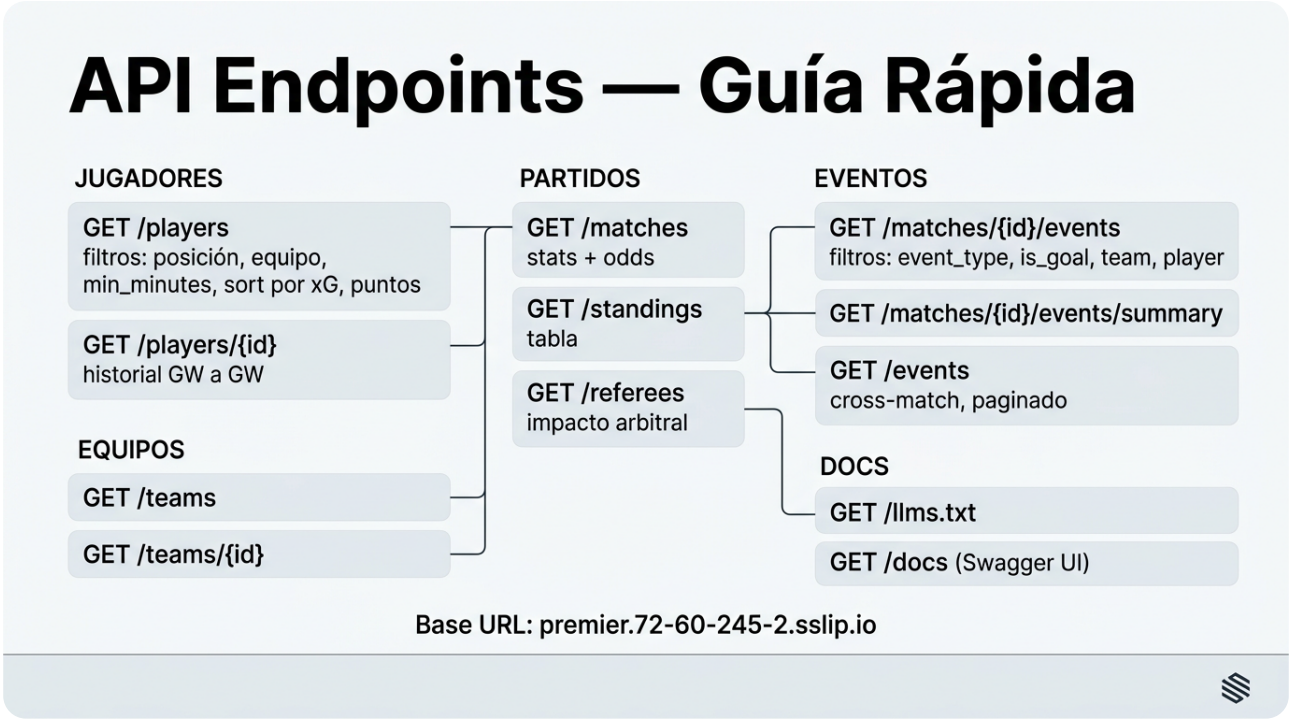
La API integra tres fuentes complementarias, cada una respondiendo una pregunta diferente:

Fuente	Pregunta	Registros	Granularidad
FPL API	¿**QUIÉN** rinde?	822 jugadores, 20 equipos, ~15K registros GW-by-GW	Por jugador
football-data.co.uk	¿**QUÉ** pasó?	291 partidos con stats + odds de apuestas	Por partido
WhoScored	¿**CÓMO** pasó?	444,252 eventos con coordenadas x,y	Por acción

Datos Disponibles (temporada 2025-26)

Dato	Cantidad
Jugadores	822 (90+ campos cada uno: xG, xA, ICT, form, etc.)
Equipos	20 (con strength ratings local/visitante)
Partidos	291 de 380 (temporada en curso)
Gameweeks	38 (con deadlines y promedios)
Historial por jugador	~15,000 registros GW-by-GW
Eventos (WhoScored)	444,252 (pases, tiros, tackles, goles...)
Goles registrados	807
Goleadores	Haaland (22), Igor Thiago (18), Semenyo (15)

Endpoints Disponibles



15 endpoints REST organizados por categoría: jugadores, equipos, partidos y eventos

Jugadores (FPL API)

Endpoint	Descripción	Parámetros
`GET /players`	Lista de jugadores con filtros	`position`, `team`, `min_minutes`, `sort_by`, `limit`
`GET /players/{id}`	Detalle + historial GW-by-GW	—

Campos de ordenamiento (sort_by): `total_points`, `goals_scored`, `assists`, `expected_goals`, `expected_assists`, `ict_index`, `minutes`, `now_cost`, `form`, `selected_by_percent`, `bonus`, `bps`, `clean_sheets`

Equipos (FPL API)

Endpoint	Descripción
`GET /teams`	20 equipos con strength ratings
`GET /teams/{id}`	Equipo con plantilla completa

Partidos (football-data.co.uk)

Endpoint	Descripción	Parámetros
`GET /matches`	Resultados + stats + odds	`team`, `result`, `limit`
`GET /standings`	Tabla de posiciones (calculada)	—
`GET /referees`	Estadísticas de impacto arbitral	—

Eventos (WhoScored)

Endpoint	Descripción	Parámetros
`GET /matches/{id}/events`	Eventos de un partido	`event_type`, `team`, `player`, `is_goal`, `is_shot`, `period`
`GET /matches/{id}/events/summary`	Conteo por tipo de evento	—
`GET /events`	Consulta cross-match	`event_type`, `team`, `player_name`, `is_goal`, `is_shot`, `limit` (max 5000)

Primeros Pasos en Python

Setup básico

```
import requests
import pandas as pd

BASE = "https://premier.72-60-245-2.sslip.io"
```

Ejemplo 1: Cargar todos los jugadores

```
resp = requests.get(f"{BASE}/players?limit=500&min_minutes=0")
players = resp.json()["players"]
df_players = pd.DataFrame(players)

print(f"Jugadores: {len(df_players)}")
print(df_players[["web_name", "team", "goals_scored",
                  "expected_goals", "minutes"]].head(10))
```

Ejemplo 2: Tabla de posiciones

```
resp = requests.get(f"{BASE}/standings")
standings = resp.json()["standings"]
df_standings = pd.DataFrame(standings)
print(df_standings.to_string(index=False))
```

Ejemplo 3: Todos los partidos para ML

```
resp = requests.get(f"{BASE}/matches?limit=500")
matches = resp.json()["matches"]
df_matches = pd.DataFrame(matches)

# Features para predicción de resultado
X = df_matches[["hs", "as_", "hst", "ast", "hf", "af",
                "hc", "ac", "b365h", "b365d", "b365a"]]
y = df_matches["ftr"]  # Target: H (Home), D (Draw), A (Away)

print(f"Partidos: {len(df_matches)}")
print(f"Distribución: {y.value_counts().to_dict()}")
```

Ejemplo 4: Eventos de un partido

```
# Eventos del partido #1 (Liverpool vs Bournemouth)
resp = requests.get(f"{BASE}/matches/1/events")
data = resp.json()
print(f"{data['home_team']} vs {data['away_team']}")
print(f"Total eventos: {data['total_events']}")

df_events = pd.DataFrame(data["events"])
print(df_events["event_type"].value_counts().head(10))
```

Ejemplo 5: Todos los goles de la temporada

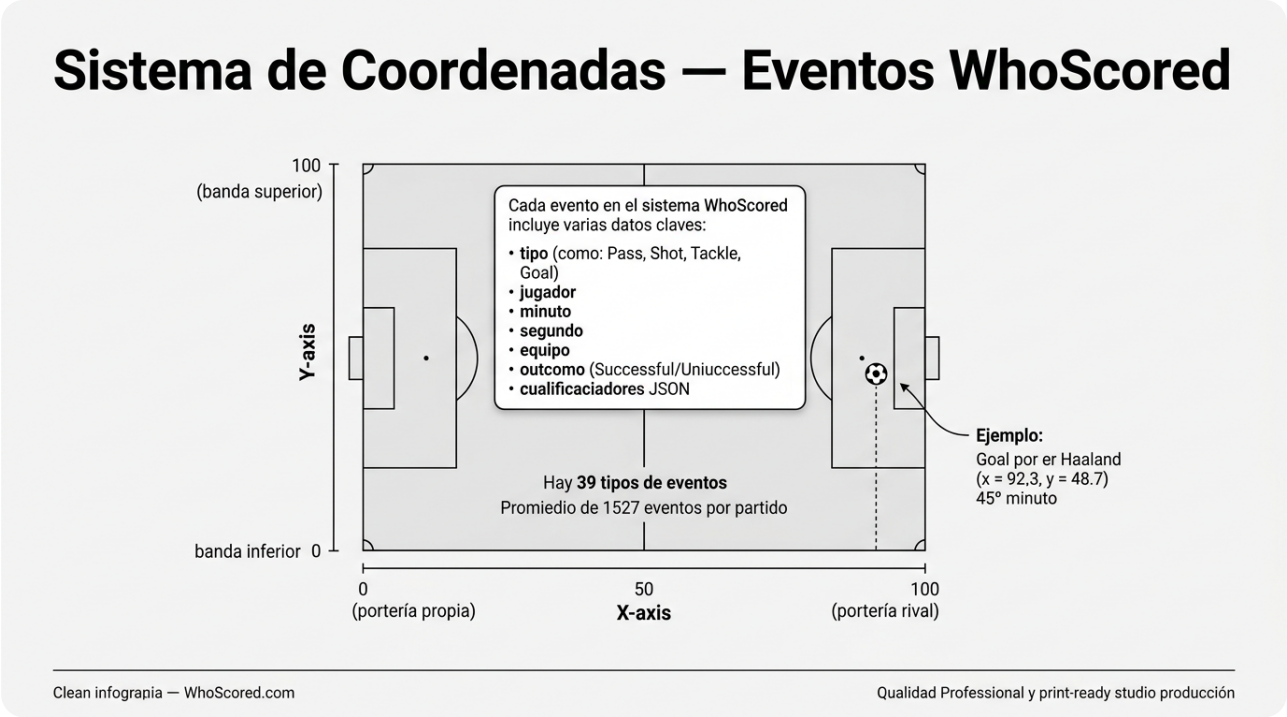
```
resp = requests.get(f"{BASE}/events?is_goal=true&limit=1000")
goals = resp.json()["events"]
df_goals = pd.DataFrame(goals)

# Top goleadores
print(df_goals["player_name"].value_counts().head(10))

# Distribución de goles por zona del campo
print(f"x promedio: {df_goals['x'].mean():.1f}")
print(f"y promedio: {df_goals['y'].mean():.1f}")
```

Datos de Eventos (WhoScored)

La fuente más granular de la API. Cada acción en cada partido está registrada con coordenadas x,y, tipo, resultado y qualifiers detallados.



Sistema de coordenadas: x (0-100) de portería propia a rival, y (0-100) de banda a banda

Sistema de Coordenadas

- **Eje X:** 0 = portería propia del equipo → 100 = portería rival
- **Eje Y:** 0 = banda inferior → 100 = banda superior
- Cada equipo ataca de izquierda a derecha (x=0 a x=100)

Tipos de Evento (39 tipos)

Tipo	Cantidad	% del Total	Descripción
`Pass`	278,502	62.7%	Pases (incluye corners, tiros libres)
`BallRecovery`	23,544	5.3%	Recuperaciones de balón
`BallTouch`	20,521	4.6%	Toques de balón
`Aerial`	18,744	4.2%	Duelos aéreos
`Clearance`	16,401	3.7%	Despejes
`Foul`	12,618	2.8%	Faltas
`TakeOn`	10,450	2.4%	Regates/dribblings
`Tackle`	9,815	2.2%	Entradas
`Goal`	807	0.2%	Goles

Estructura de un Evento

```
{
  "id": 7018,
  "match_id": 1,
  "minute": 36,
  "second": 16,
  "period": "FirstHalf",
  "event_type": "Goal",
  "outcome": "Successful",
  "team_name": "Liverpool",
  "player_name": "Hugo Ekitiké",
  "x": 89.1,
  "y": 52.4,
  "goal_mouth_y": 48.0,
  "goal_mouth_z": 3.2,
  "is_goal": true,
  "is_shot": true,
  "qualifiers": [
    {"type": {"displayName": "LowRight"}},
    {"type": {"displayName": "BigChance"}},
    {"type": {"displayName": "RegularPlay"}}
  ]
}
```

Qualifiers Importantes

Qualifier	Significado
`BigChance`	Ocasión clara de gol (expected conversion > 35%)
`KeyPass`	Pase que crea una oportunidad de gol
`Corner`	Evento desde corner
`Cross`	Centro al área
`LongBall`	Pase largo
`ThroughBall`	Pase filtrado
`Head`	Acción con la cabeza
`RightFoot` / `LeftFoot`	Pie utilizado

Qualifier	Significado
`RegularPlay` / `SetPiece`	Tipo de jugada

Recetas para Machine Learning

Mapa de tiros (Shot Map)

```
import matplotlib.pyplot as plt

# Obtener todos los tiros
shots = requests.get(
    f"{BASE}/events?is_shot=true&limit=5000"
).json()["events"]
df = pd.DataFrame(shots)

fig, ax = plt.subplots(figsize=(12, 8))
goals = df[df["is_goal"] == True]
misses = df[df["is_goal"] == False]

ax.scatter(misses["x"], misses["y"], alpha=0.3, s=20, c="gray")
ax.scatter(goals["x"], goals["y"], s=80, c="red", marker="*",
           zorder=5, label=f"Goles ({len(goals)})")
ax.set_xlabel("x (0=portería propia → 100=portería rival)")
ax.set_ylabel("y (0=banda inferior → 100=banda superior)")
ax.legend()
plt.title("Shot Map — Premier League 2025-26")
plt.show()
```

Comparar pases exitosos vs fallidos por equipo

```
# Pases de Arsenal
passes = requests.get(
    f"{BASE}/events?event_type=Pass&team=Arsenal&limit=5000"
).json()["events"]
df = pd.DataFrame(passes)

success = df[df["outcome"] == "Successful"]
fail = df[df["outcome"] == "Unsuccessful"]
print(f"Arsenal: {len(success)} exitosos / {len(fail)} fallidos")
print(f"Precisión: {100*len(success)/len(df):.1f}%")
```

Features derivadas para predicción

```
# Por cada partido: contar eventos por tipo
import numpy as np

all_matches = requests.get(f"{BASE}/matches?limit=500").json()
features = []

for m in all_matches["matches"][:50]:  # Primeros 50
    summary = requests.get(
        f"{BASE}/matches/{m['id']}/events/summary"
    ).json()

    features.append({
        "match_id": m["id"],
        "result": m["ftr"],
        "home_passes": summary["event_types"].get("Pass", 0),
        "home_shots": m["hs"],
        "home_tackles": summary["event_types"].get("Tackle", 0),
    })

df_feat = pd.DataFrame(features)
```

Métricas de Rendimiento

Endpoint	Tiempo respuesta	Registros
`/players` (10 items)	~500ms	822 disponibles
`/matches` (10 items)	~500ms	291 disponibles
`/standings`	~500ms	20 equipos
`/matches/{id}/events`	~1.4s	~1,527/partido
`/events?is_goal=true` (50)	~1.0s	807 goles
`/health`	~500ms	Status completo

Latencia: ~500ms para endpoints simples, ~1-1.5s para eventos (tabla de 444K registros). La API corre en un VPS dedicado con SSL/TLS.

Validación de Datos

Los datos han sido validados contra fuentes oficiales:

Validación	Resultado
Cobertura de partidos	291/291 (100%) con eventos
Goles eventos vs score	0 discrepancias
Campos NULL en eventos	0 (event_type, minute, team, x, y, period)
Coordenadas fuera de rango	0 (todas en 0-100)
Eventos duplicados	0
Top goleador	Haaland (22) — coincide con FPL API

Tabla de Posiciones (verificada al 15/03/2026)

#	Equipo	PJ	G	E	P	GF	GA	Pts
1	Arsenal	30	20	7	3	59	22	**67**
2	Man City	29	18	6	5	59	27	**60**
3	Man United	29	14	9	6	51	40	**51**
4	Aston Villa	29	15	6	8	39	34	**51**
5	Chelsea	29	13	9	7	53	34	**48**

Insights Clave para ML

- **Mejor predictor:** Diferencial de tiros al arco (SOTDiff) — correlación +0.547 con resultado
- **Distribución de resultados:** Local 42.3% | Empate 26.1% | Visitante 31.6%
- **Benchmark Bet365:** 50.2% de acierto en 3 clases (H/D/A) — un modelo ML que supere esto compite con las casas de apuestas
- **Corners NO predicen resultado** — correlación +0.014
- **Árbitros importan:** Home win % varía de 11.8% (Gillett) a 71.4% (Bankes)

- **Promedio goles/partido:** 2.77 — Over 2.5 goles en 54% de partidos

---

Troubleshooting

Problema	Solución
`ConnectionError`	Verificar internet. La API no requiere VPN
Datos vacíos	Verificar que el `id` existe con `/matches` o `/players`
Muchos requests	Usar `limit` para paginar. Max 5000 en `/events`
JSON anidado	`pd.DataFrame(resp.json()["players"])` — acceder al key correcto
Qualifiers como string	`json.loads(event["qualifiers"])` si llega como string

Premier League Data API v2.0 — Datos actualizados diariamente a las 06:00 UTC
Machine Learning I (ML1-2026I) — Universidad Externado de Colombia